

Rapport TP 9

Vincent PLESSY

9 avril 2025

1 NOTA BENE

voir dessin.pdf pour avoir des dessins illustrant mes fonctions ,c'est compliqué de tout mettre dans le rapport latex

2 Introduction

L'objectif de ce projet est de mettre en place un allocateur de mémoire fonctionnant sur un pool statique. Le but initial était d'implémenter un allocateur robuste, capable d'allouer et de libérer de la mémoire à partir d'un grand segment mémoire donné par le prof, tout en maintenant une gestion correcte des blocs. Par la suite, il a été nécessaire d'optimiser la rapidité de l'allocateur et de limiter la fragmentation. N'ayant pu commenter le code car la norme ISO C90 me l'empêche, je vais essayer de bien expliquer dans ce rapport.

3 Mes objectifs

- Il fallait faire un **allocateur/libérateur** qui ne plante pas , me permettant une alloc fiable
- Il faudra réduire la fragmentation interne voire externe en découpant mon bloc.
- Il faudra également **accélérer** le processus d'allocation en évitant de toujours repartir au début de la liste des blocs libres .

4 Étapes de raisonnement

Avant que j'ajoute les améliorations, mes étapes de réflexion ont été les suivantes :

1. **Besoin de robustesse** : Le premier objectif était que l'allocateur puisse allouer et libérer la mémoire sans erreur. J'ai donc conçu le système avec un pool de mémoire statique et une liste doublement chaînée de blocs, où chaque bloc possède un entête contenant la taille, un indicateur d'occupation et des pointeurs vers les blocs adjacents. Il aura fallu que je crée une structure adaptée pour cela :

```
typedef struct bloc{
    size_t size;
    int free;
    struct bloc *suiv;
    struct bloc *prec;

}bloc;
```

2. **Gestion de la fragmentation** : L'utilisation de la fonction `fragment()` permet de découper un bloc trop grand en deux, de façon à fournir exactement la taille demandée tout en restituant le reste comme bloc libre. Cela aide à réduire la fragmentation interne.(voir scan ci-joint)
3. **Optimisation de la rapidité** : Plutôt que de parcourir systématiquement comme un gros goret l'intégralité de la liste des blocs libres, j'ai préféré prendre un pointeur global `last_free`. Ce pointeur conserve le dernier bloc libre trouvé lors d'une allocation, de sorte que la recherche suivante démarre à partir de ce point . Cela permet de réduire le nombre d'itérations nécessaires, surtout lorsque la liste devient longue.

5 Améliorations apportées

Voici les deux grandes améliorations intégrées dans mon code :

1. Découpage - via la fonction `fragment`

- **Objectif** : Ne réserver que la taille demandée et remettre le surplus en bloc libre, afin de limiter la fragmentation interne.
- **Implémentation** : La fonction `fragment(b, new_size)` vérifie que le bloc actuel `b` possède un espace suffisant pour être divisé :

```
if (b->size >= new_size + EN_TETE + 1)
```

 Si c'est le cas, le bloc `b` est découpé : le nouveau bloc (`nouv`) est créé à l'adresse calculée par addition de `EN_TETE` et `new_size` à l'adresse de `b`. Ensuite, les tailles et les pointeurs (`suiv` et `prec`) sont mis à jour.
- **Avantage** : La taille effective allouée correspond exactement à la demande, et l'espace excédentaire est récupéré pour de futures allocations.

2. Optimisation de la rapidité avec `last_free`

- **Objectif** : Éviter de toujours recommencer la recherche du bloc libre à partir du début de la liste, ce qui peut être coûteux en temps.
- **Implémentation** : On va utiliser le pointeur `last_free` est utilisé pour mémoriser le dernier bloc libre où s'est terminée la dernière allocation. Lors d'un appel à `my_malloc`, la recherche du bloc libre commence à partir de `last_free` (ou de `list`

si `last_free` n'est pas initialisé), plutôt que de repartir au début du bloc. Après l'allocation, `last_free` est mis à jour pour pointer sur le bloc suivant (ou sur le début de la liste si le bloc actuel est le dernier).

- **Avantage :** Cette stratégie diminue le nombre d'itérations dans la liste, en particulier dans un scénario à forte charge où de nombreuses allocations et libérations surviennent.

6 Explication `my_malloc`

ma fonction commence par vérifier que la taille demandée n'est pas zéro. Si le pool n'est pas initialisé, on appelle `cree_memoire()`. Elle utilise le pointeur `last_free` pour démarrer la recherche d'un bloc libre, ce qui réduit le temps de parcours en cas de nombreuses allocations :

- Le `do while` parcourt la liste des blocs à partir de `start` et recherche un bloc libre suffisamment grand. Si un bloc est trop grand, on appelle la fonction `fragment` pour le découper. Le bloc trouvé est marqué comme occupé et l'adresse retournée est celle située après l'en-tête, avec :

```
return (void*)((char*)tmp + EN_TETE);
```

- En convertissant en `char*` `tmp`, chaque incrément avance de 1 octet, ce qui permet de manipuler la mémoire avec une précision au niveau de l'octet. Ensuite on le transforme en `void*` pour correspondre avec le type de retour de la fonction.
- lors du prochain appel de notre fonction qui sera effectué par le main donné par le prof, on repartira directement du dernier bloc libre trouvé, au lieu de revenir à chaque fois au début de la liste
- si aucun bloc n'est trouvé, la fonction renvoie `NULL`.

7 Explication `my_free`

ma fonction commence par vérifier que le pointeur fourni n'est pas `NULL` :

- On récupère l'en-tête du bloc associé en soustrayant `EN_TETE` de l'adresse donnée :

```
bloc *nouv=(bloc*)((char*)pouet - EN_TETE);
```

- Ensuite on fusionne le bloc libre avec le bloc suivant s'il est aussi libre, en augmentant la taille du bloc courant avec la taille (plus en-tête) du bloc suivant, et en ajustant les pointeurs.

```
if (nouv->suiv && nouv->suiv->free){
nouv->size += EN_TETE + nouv->suiv->size;
nouv->suiv=nouv->suiv->suiv;
```

```
if (nouv->suiv){
nouv->suiv->prec=nouv;
```

```
}
```

- Ensuite,même principe , on fusionne avec le bloc precedent s'il est libre,afin de reduire la fragmentation interne.

```
    if (nouv->prec && nouv->prec->free){  
        nouv->prec->size += EN_TETE + nouv->size;  
        nouv->prec->suiv=nouv->suiv;  
        if (nouv->suiv){  
            nouv->suiv->prec=nouv->prec;  
        }  
        nouv=nouv->prec;  
    }
```

- on met ensuite à jour last_free avec le bloc fusionné , pour optimiser lors des prochains appels

```
    last_free=nouv;
```